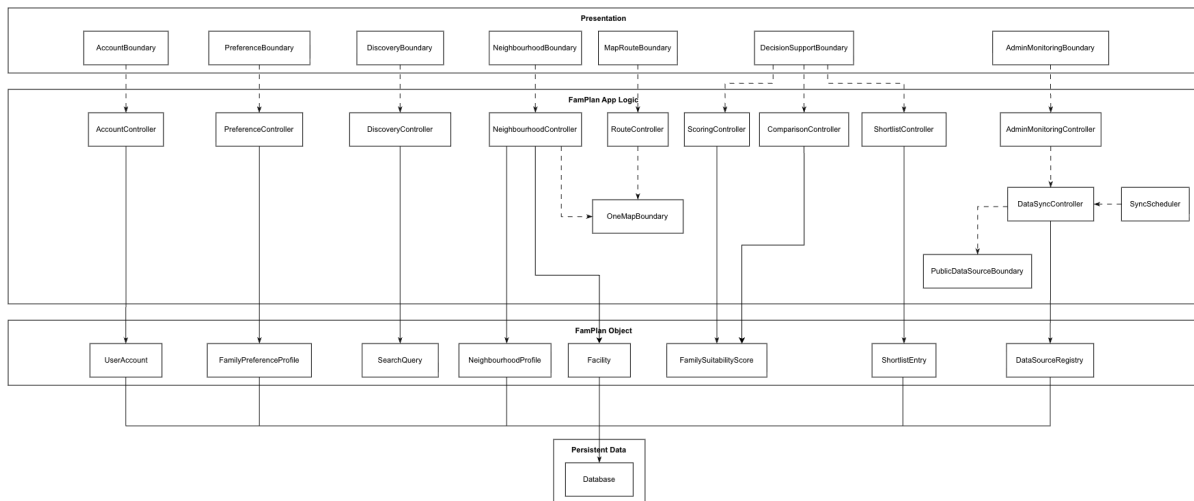




Presentation Layer

This layer is responsible for user interaction and view rendering inside the user's web browser, implemented as a TypeScript Single-Page Application using React and Vite. The boundary components capture user inputs, enforce form formatting, and delegate actions to backend REST API endpoints.

This layer consists of:

1. **AccountBoundary**: Presents registration, login, logout, display name updates, and password reset forms. Captures credentials and displays operation outcomes to users.
2. **PreferenceBoundary**: Captures family life stages, preferred planning areas, priority commute destinations, and 0–5 criterion importance weights.
3. **DiscoveryBoundary**: Captures neighbourhood search queries and filter options (amenity distances, counts, and commute limits). Presents filterable and sortable result cards.
4. **NeighbourhoodBoundary**: Displays neighbourhood summary profiles, amenity counts by category, and availability states.
5. **MapRouteBoundary**: Renders an interactive Leaflet map marking categorized facilities, proximity radii (500 m, 1 km, 2 km), and travel route estimates.
6. **DecisionSupportBoundary**: Presents suitability score breakdowns, captures comparison selections (2 to 4 neighbourhoods), and displays the user's shortlisted neighbourhoods and private notes.
7. **AdminMonitoringBoundary**: Displays the automated synchronisation dashboard showing data source health, record counts, execution timestamps, and sync logs.

App Logic Layer

This layer contains all controller classes and external adapter boundaries executing on the Node.js / Express backend server. It coordinates business rules, processes data transformations, and encapsulates third-party integrations.

This layer consists of:

1. **AccountController**: Enforces password hashing using Argon2id, verifies session tokens, enforces login lockout policies (15 minutes after 5 failures), and manages account deletion lifecycles.
2. **PreferenceController**: Evaluates family planning stages, supplies stage-based weight presets, and manages persistent user preference profiles.
3. **DiscoveryController**: Validates search query bounds and executes spatial and attribute filters over cached neighbourhood datasets.
4. **NeighbourhoodController**: Aggregates nearby facilities within requested radius buffers and compiles the amenity summary for neighbourhood profiles.
5. **RouteController**: Validates transit modes and calculates commute estimates between neighbourhood reference points and priority destinations.
6. **ScoringController**: Implements the deterministic scoring engine, normalizes 8 criterion values, applies missing-data policies, and computes the rounded 0–100 suitability score.
7. **ComparisonController**: Validates that 2 to 4 neighbourhoods are selected and compiles side-by-side metric evaluations.
8. **ShortlistController**: Enforces the 10-entry shortlist capacity limit per user and validates private text notes up to 500 characters.
9. **AdminMonitoringController**: Enforces administrator role authorization and retrieves data source health and synchronisation audit logs.
10. **DataSyncController**: Fetches datasets from external endpoints, validates dataset schemas against checksums, creates provenance stamps, and persists validated snapshots.
11. **SyncScheduler**: A background service powered by node-cron that automatically executes scheduled data synchronisation tasks at least once every 24 hours without human intervention.
12. **OneMapBoundary**: External adapter interfacing with the Singapore OneMap REST API for address geocoding, reverse geocoding, and transit routing.
13. **PublicDataSourceBoundary**: External adapter encapsulating public data providers (ECDA childcare, MOE schools, URA subzones, LTA DataMall) to insulate domain logic against schema changes.

App Object Layer

This layer consists of the core domain entity classes depicted in the architecture diagram:

1. **UserAccount**: Represents a registered user identity and encapsulates account status, timestamps, and role privileges.
2. **FamilyPreferenceProfile**: Stores user-configured family planning stages, preferred planning areas, and amenity distance criteria.
3. **SearchQuery**: Encapsulates transient request criteria, location text, and active filter thresholds during neighbourhood discovery.
4. **NeighbourhoodProfile**: Represents a comparable planning zone with name, planning area, reference point coordinates, and boundary basis.
5. **Facility**: Represents an individual school, clinic, preschool, supermarket, park, or transport station with geocoordinates and source metadata.
6. **FamilySuitabilityScore**: Encapsulates the overall 0–100 score, calculation configuration version, and contributing criterion breakdowns.

7. **ShortlistEntry**: Represents a saved neighbourhood pinned to a user account, supporting an optional private note.
8. **DataSourceRegistry**: Maintains configuration, freshness thresholds, and operational health status for each public data source.

Persistent Data Layer

This layer contains the database that will store all of the entities. In FamPlan, this is implemented using a PostgreSQL relational database to reliably manage and persist application data.